

CSCI 570 — Final Project Summary

Sequence Alignment: Basic DP and Memory-Efficient (Hirschberg) Solutions

1. Project Overview

This project implements two solutions to the Sequence Alignment problem as described in Kleinberg & Tardos:

- **Basic Dynamic Programming** — the standard $O(mn)$ time, $O(mn)$ space algorithm.
- **Memory-Efficient (Hirschberg-style)** — a divide-and-conquer approach that combines linear-space DP with the full DP traceback, achieving $O(mn)$ time with only $O(m+n)$ space.

Both programs accept an input file (containing base strings and insertion indices that generate the actual strings) and an output file path. They produce:

1. Minimum alignment cost (integer)
2. Aligned first string (with `_` for gaps)
3. Aligned second string (with `_` for gaps)
4. CPU time in milliseconds (float)
5. Memory usage in kilobytes (float)

The gap penalty is $\delta = 30$, and mismatch costs α are fixed per the project specification.

2. Basic Dynamic Programming Algorithm

The basic algorithm fills an $(m+1) \times (n+1)$ DP table where `dp[i][j]` represents the minimum alignment cost of the first i characters of X with the first j characters of Y .

Recurrence:

```
dp[i][j] = min {
    dp[i-1][j-1] +  $\alpha(x_i, y_j)$ ,
    dp[i-1][j] +  $\delta$ ,
    dp[i][j-1] +  $\delta$ 
}
```

Base cases: `dp[i][0] = i ·  $\delta$` and `dp[0][j] = j ·  $\delta$` .

After filling the table bottom-up, a top-down traceback reconstructs the optimal alignment by following the decisions that led to each cell's value.

	Time	Space
--	------	-------

Basic DP	$O(mn)$	$O(mn)$
----------	---------	---------

The $O(mn)$ space requirement comes from storing the full DP table, which is needed for the traceback step.

3. Memory-Efficient (Hirschberg-style) Algorithm

The memory-efficient algorithm uses divide-and-conquer combined with a linear-space forward pass of the DP to reduce memory from $O(mn)$ to $O(m+n)$ while maintaining $O(mn)$ time.

Key Components

Linear-space forward pass (`linear_bottom_up`): Computes the last row of the DP table using only two rows (previous and current), requiring $O(n)$ space.

Divide step:

1. Split X at its midpoint: `x_left = X[:mid]` , `x_right = X[mid:]` .
2. Run the linear forward pass on `(x_left, Y)` to get forward costs.
3. Run the linear forward pass on `(x_right reversed, Y reversed)` to get backward costs.
4. Find the optimal split point q^* in Y that minimizes `forward[q] + backward[n - q]` .

Conquer step: Recursively align `(x_left, Y[:q*])` and `(x_right, Y[q*:])` , then concatenate.

Base case: When the subproblem is small enough (`len(x) * len(y) ≤ 2,000,000`), fall back to the full DP with traceback.

	Time	Space
Memory-Efficient	$O(mn)$	$O(m+n)$

The time remains $O(mn)$ because the sum of subproblem sizes across all recursion levels is bounded by $2mn$ (each level halves X while scanning all of Y). The space is $O(m+n)$ because only two rows of the DP table are ever in memory, plus $O(m)$ recursion depth.

4. Datapoints Output Table

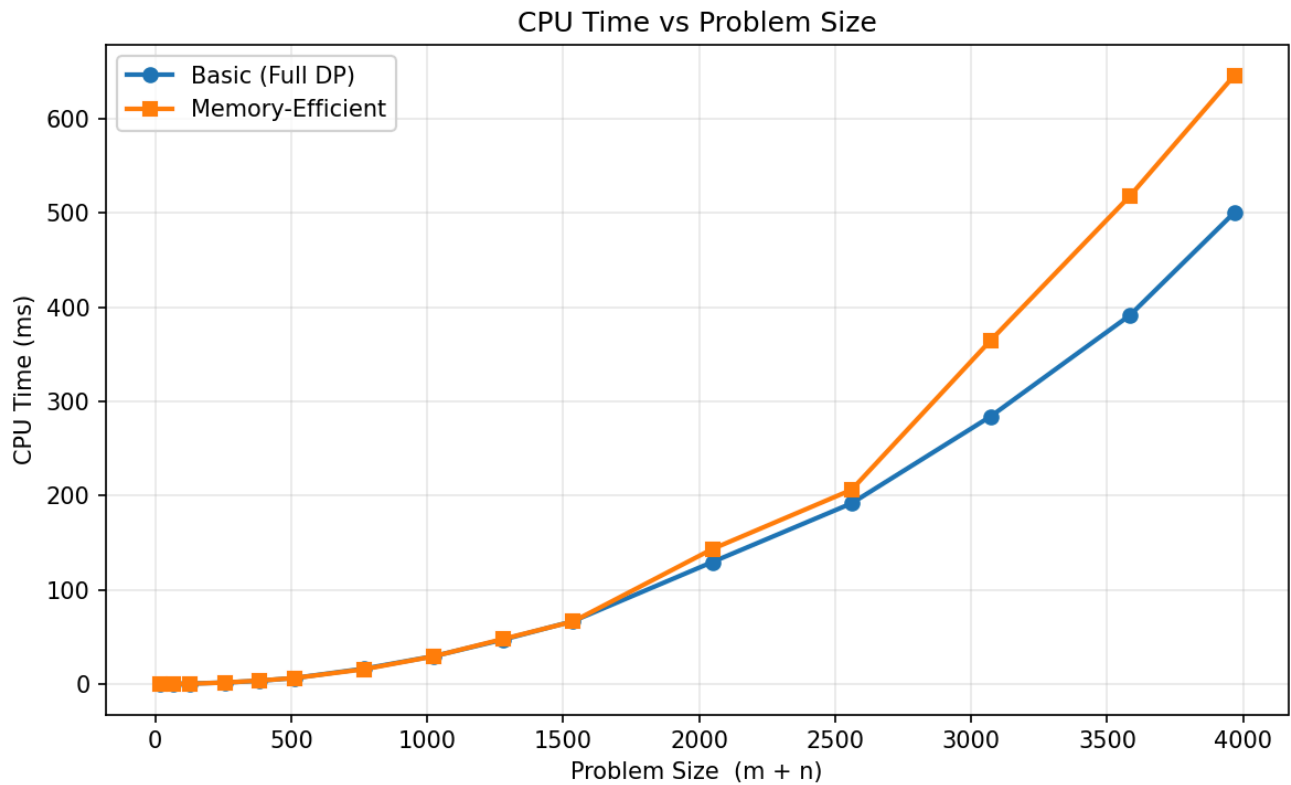
Benchmarks were run on 15 input files with increasing problem sizes ($m+n$):

Input	Problem Size ($m+n$)	Basic Time (ms)	Basic Memory (KB)	Efficient Time (ms)	Efficient Memory (KB)
in1	16	0.14	18,976	0.02	19,200
in2	64	0.23	19,008	0.12	19,264
in3	128	0.52	19,136	0.39	19,552

in4	256	1.63	19,664	1.62	19,904
in5	384	3.55	20,640	3.69	20,928
in6	512	6.41	21,760	6.39	20,960
in7	768	16.32	25,408	15.78	21,424
in8	1,024	29.58	30,112	29.43	23,040
in9	1,280	47.25	36,032	47.93	23,904
in10	1,536	66.74	43,360	66.37	26,144
in11	2,048	129.45	63,120	143.26	31,568
in12	2,560	191.77	87,232	206.16	37,280
in13	3,072	284.14	115,440	364.88	30,448
in14	3,584	391.49	149,008	518.08	34,352
in15	3,968	500.45	174,880	646.09	28,128

5. Graphs and Insights

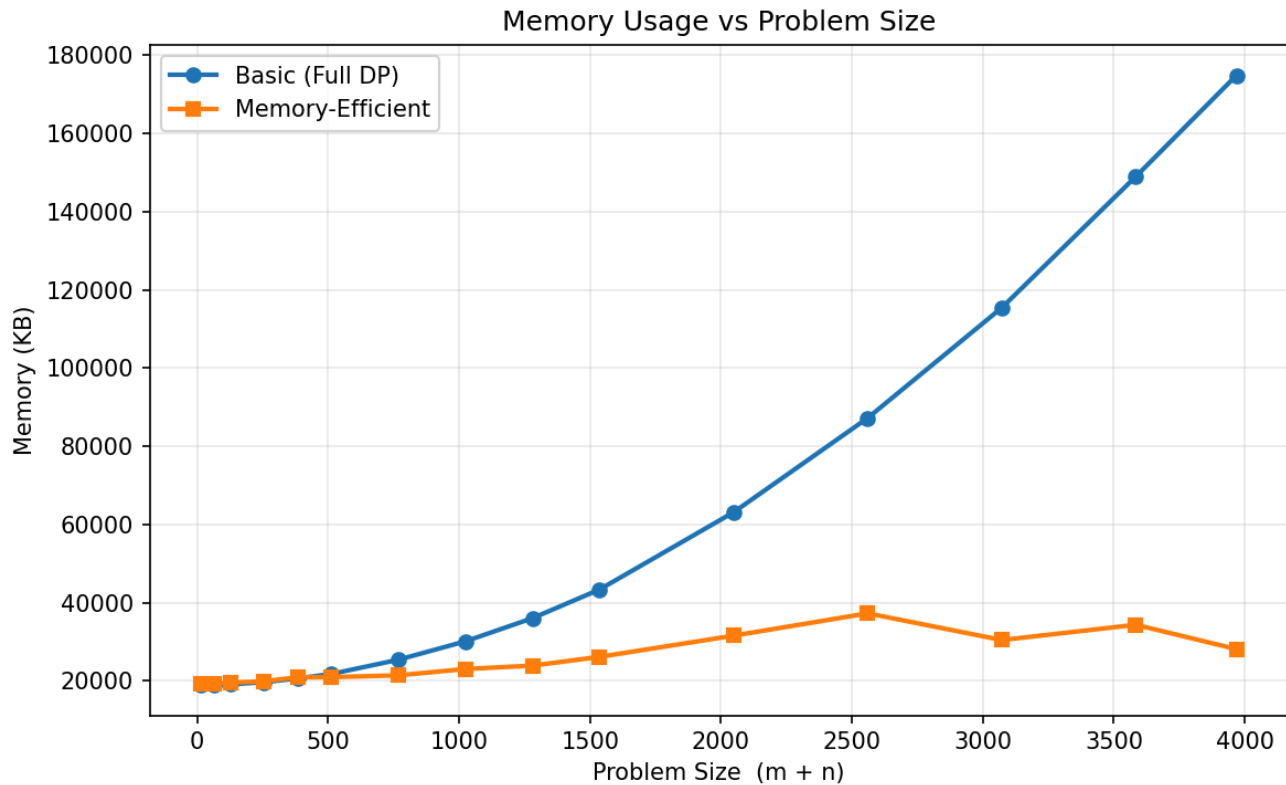
5.1 CPU Time vs Problem Size



Insights:

- Both algorithms exhibit $O(mn) \approx O(n^2)$ time growth, as expected. The curves are roughly quadratic.
- For small problem sizes (up to $\sim 1,500$), the two algorithms perform nearly identically in time.
- At larger problem sizes, the memory-efficient algorithm is approximately $1.2\text{--}1.3\times$ slower than the basic algorithm. This overhead comes from: (a) running the linear-space DP twice per recursion level (forward and backward), and (b) the recursion and subproblem management overhead.
- The constant-factor slowdown is the expected trade-off for the dramatic memory savings.

5.2 Memory Usage vs Problem Size



Insights:

- The basic algorithm's memory grows quadratically with problem size — consistent with $O(mn)$ space. At the largest input ($m+n = 3,968$), it consumes ~ 175 MB.
- The memory-efficient algorithm's memory stays roughly flat between 19–37 MB regardless of problem size — consistent with $O(m+n)$ linear space.
- At the largest problem size, the basic algorithm uses approximately **6× more memory** than the efficient version (174,880 KB vs 28,128 KB).
- This distinction would become even more dramatic for larger inputs (e.g., strings of length 10,000+), where the basic algorithm would require gigabytes of memory while the efficient algorithm remains feasible.

6. Optimal Trivial Problem Size

The memory-efficient algorithm falls back to the full DP (with traceback) when the subproblem is small enough. In our implementation, the threshold is:

$$\text{len}(x) * \text{len}(y) \leq 2,000,000$$

This means subproblems with at most ~2 million cells use the basic DP directly, avoiding further recursion overhead.

Rationale:

- A 2M-element integer array in Python consumes approximately 16–60 MB depending on the Python list overhead, which is acceptable for a single subproblem.
- Below this threshold, the basic DP is faster due to no recursion overhead and only a single table fill + traceback pass.
- Above this threshold, the memory savings from the divide-and-conquer approach outweigh the additional time cost of running forward/backward passes.

Impact on runtime: Setting the trivial size too low increases recursion depth and the number of linear-space passes, adding constant-factor overhead. Setting it too high increases peak memory usage. The chosen threshold of 2,000,000 cells provides a good balance — keeping the efficient algorithm’s memory under ~40 MB even for the largest benchmark inputs, while limiting the time overhead to roughly 1.3× compared to the basic version.

7. Team Contributions

Team Member	Contributions
Raunak Sood	Testing and verification of both algorithms on sample and benchmark inputs; benchmark result review and analysis; final report preparation and formatting